MIPS nRisc de 8 *bits*

Kayky Moreira Praxedes, Carlos Ernesto Cardoso dos Reis

Março 2026

1 Objetivo do projeto

Construção e simulação de um **processador *MIPS nRisc* monociclo**, projetado integralmente em Verilog HDL, com arquitetura simplificada de 8 *bits* para dados, instruções e memória [01].

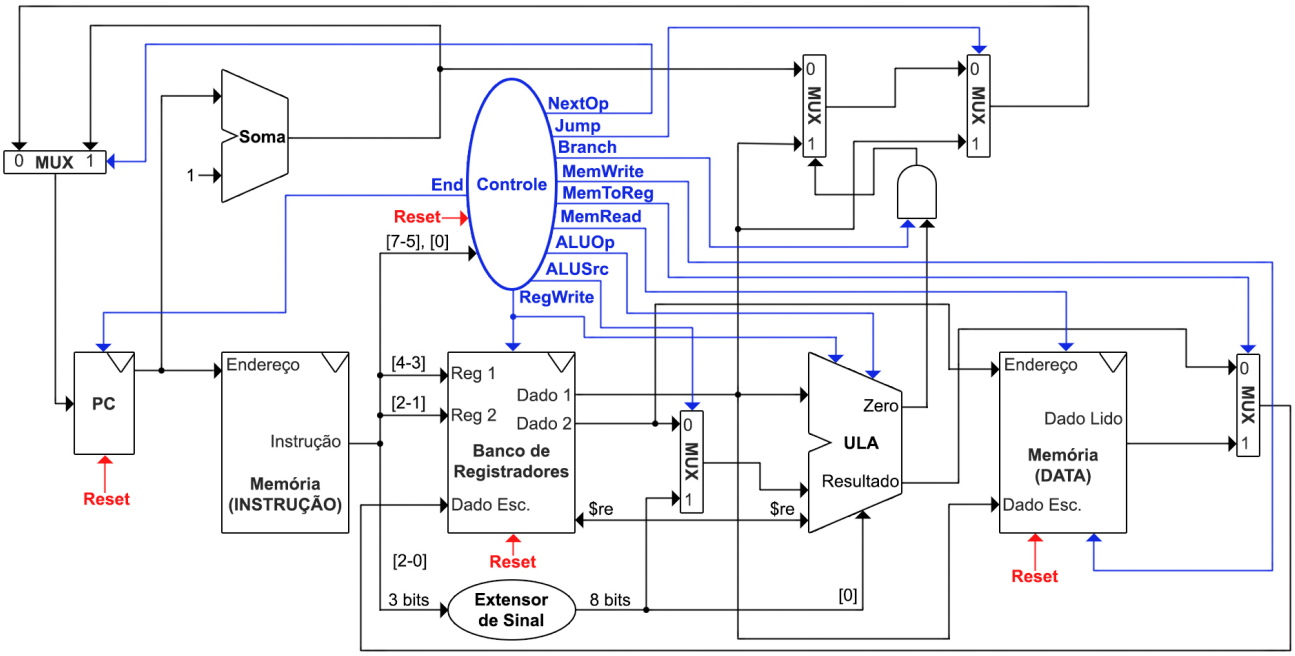
Programa embarcado:

Foi embarcado na memória de instruções um programa que executa a Cifra de César sobre uma mensagem armazenada na memória de dados (verificar o apêndice C). O algoritmo:

1. Lê um caractere da memória.
2. Soma o deslocamento (chave, sendo número entre -15 e 15).
3. Ajusta o valor caso ultrapasse o intervalo alfabético.
4. Armazena o novo caractere na memória.

O limite da mensagem é de 128 caracteres, imposto pelo espaço de endereçamento (o mesmo vale para o tamanho do programa).

2 Diagrama



Caminho dos dados:

Os dados seguem uma ordem fixa:

1. *PC* acessa memória de instruções
2. Instrução é decodificada pela unidade de controle
3. Registradores são lidos
4. ULA executa operação
5. Resultado é escrito no banco ou memória

3 Arquitetura

Quais são os principais componentes e como eles funcionam (verificar o apêndice B).

1 Unidade de controle:

Geração dos sinais de controle a partir do *OpCode* e do campo *funct* (determinam o fluxo do caminho de dados e a operação realizada em cada ciclo).

Flags do Controle (binário):

	0000 add	0001 sub	001X addi	0100 mult	0101 null	0110 ld	0111 st	1000 jr	1001 null	1010 beq	1011 slt	1100 result	1101 null	1110 nop	1111 hlt
ALUSrc	0	0	1	0	0	0	0	0	0	0	0	1	1	1	1
MemToReg	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
RegWrite	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0
MemRead	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0
MemWrite	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
Branch	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
Jump	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
NextOp	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
End	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
ALUOp	00	00	01	10	10	01	01	01	01	11	11	00	00	01	01

- `ALUSrc`: Indica se o segundo operando vem de um registrador (0), ou se trata de um imediato (1).
- `MemToReg`: Indica se o resultado a ser escrito no registrador vem da ULA (0) ou se vem da memória (1).
- `RegWrite`: Indica se vai haver escrita no registrador (registrador 1) (1) ou não (0).
- `MemRead`: Indica se vai haver leitura de memória (1) ou não (0).
- `MemWrite`: Indica se vai haver escrita na memória (1) ou não (0).
- `Branch`: Indica se vai haver um desvio condicional (1), ou não (0)
- `Jump`: Indica se vai haver um desvio incondicional (1), ou não (0)
- `NextOp`: Ignora a linha instrução.
- `End`: Encerra o programa.
- `ALUOp`: 2 *bits* que indicam qual vai ser o tipo de operação a ser realizado pela ULA [02]:

- 00: `add`, `sub` ou `result`
- 01: `addi`
- 10: `mult`
- 11: `beq` ou `slt`

2 *PC (program counter)*:

Registrador responsável por armazenar o endereço da próxima instrução.

3 Memória:

A memória é dividida em duas regiões, ambos com endereços de 7 *bits* (128 posições) e armazenando dados de 8 *bits*, sendo elas:

- **Memória de dados:** Responsável por armazenar os dados variáveis (nesse caso a mensagem a ser codificada).
- **Memória de instruções:** Responsável por armazenar o código do programa.

4 Banco de Registradores:

Conjunto de 4 registradores de uso geral com 8 *bits* cada, sendo acessados pelas instruções por uma ordenação numérica.

- 00: $\texttt{\$c}_0$
- 01: $\texttt{\$c}_1$
- 10: $\texttt{\$c}_2$
- 11: $\texttt{\$c}_3$

Existe ainda o registrador especial `$re`, responsável por armazenar o resultado das instruções de comparação (`beq` e `slt`). Os valores assumidos podem ser 0 (comparação é falsa) ou 1 (verdadeira), assumindo por padrão o valor -4 (11111100), evitando desvios não intencionais.

5 ULA (unidade lógica e aritmética):

Responsável pelas operações aritméticas, lógicas e de comparação [03].

O projeto não utiliza descrição comportamental (operadores aritméticos de alto nível da linguagem, como `+`, `-`, `*` ou operadores relacionais) **para as operações**. Em vez disso, a ULA foi construída de forma hierárquica, a partir de componentes básicos (somador de 1 *bit*, somador de 8 e 16 *bits*), explicitando o fluxo de *carry* e a propagação de sinais, o que permite observar diretamente a implementação física do circuito.

Extensor de sinal:

Mini módulo responsável para converter os imediatos de 3 *bits* das instruções para 8 *bits* (sendo possível realizar as operações na ULA).

4 Instruções (binário e Assembly)

As instruções possuem 8 *bits* e são divididas em três formatos, a depender da quantidade de registradores utilizados na operação [04].

Tipo 2R:

Operações entre dois registradores.

$$\underbrace{OpCode}_{3\ bits}\ \underbrace{Reg1}_{2\ bits}\ \underbrace{Reg2}_{2\ bits}\ \underbrace{funct}_{1\ bit}$$

Aritméticas:

- 000 00-11 00-11 0 $\Rightarrow$ `add` $\$c_i$ $\$c_j$ $\Rightarrow$ $\$c_i = \$c_i + \$c_j$
- 000 00-11 00-11 1 $\Rightarrow$ `sub` $\$c_i$ $\$c_j$ $\Rightarrow$ $\$c_i = \$c_i - \$c_j$
- 010 00-11 00-11 0 $\Rightarrow$ `mult` $\$c_i$ $\$c_j$ $\Rightarrow$ $\$c_i = \$c_i \times \$c_j$

Memória:

- 011 00-11 00-11 0 $\Rightarrow$ `ld` $\$c_i$ $\$c_j$ $\Rightarrow$ $\$c_i$ = `memória_de_dados[`$\$c_j$`]`
- 011 00-11 00-11 1 $\Rightarrow$ `st` $\$c_i$ $\$c_j$ $\Rightarrow$ `memória_de_dados[`$\$c_j$`]` = $\$c_i$

Comparação:

- 101 00-11 00-11 0 $\Rightarrow$ `beq` $\$c_i$ $\$c_j$ $\Rightarrow$ $\$re = (\$c_i == \$c_j)\ ?\ 1 : 0$
- 101 00-11 00-11 1 $\Rightarrow$ `slt` $\$c_i$ $\$c_j$ $\Rightarrow$ $\$re = (\$c_i < \$c_j)\ ?\ 1 : 0$

Tipo 1R:

Operações com apenas um registrador.

$$\underbrace{OpCode}_{3\ bits}\ \underbrace{Reg}_{2\ bits}\ \underbrace{Imediato}_{3\ bits}$$

Aritméticos:

- 001 00-11 000-111 $\Rightarrow$ `addi` $\$c_i$ *imediato* $\Rightarrow$ $\$c_i = \$c_i + imediato$

Desvio:

- 100 00-11 000-111 $\Rightarrow$ `jr` $\$c_i$ $\Rightarrow$ $PC = \$c_i$
- 110 00-11 000-111 $\Rightarrow$ `result` $\$c_i$ *imediato* $\Rightarrow$ $PC = (\$re == imediato)\ ?\ \$c_i : PC + 1$

O imediato possui sinal e varia de -3 a 3 (101 - 011).

Tipo 0R:

Operações que não envolvem o uso de registradores.

$$\underbrace{OpCode}_{3\ bits}\ \underbrace{don't\ care}_{4\ bits}\ \underbrace{funct}_{1\ bit}$$

Controle:

- 111 0000 0 $\Rightarrow$ `nop` (não executa nenhuma ação)
- 111 0000 1 $\Rightarrow$ `hlt` (encerra o programa)

Apêndice A(características adicionais do processador)

Objetivo do projeto:

[01] - **Todas as operações são realizadas em um ciclo de *clock***, com a duração do ciclo sendo arbitrariamente definida em 10ns (como a simulação não tem um limite físico, qualquer valor seria aceito).

Arquitetura

[02] - O `ALUOp` 01 foi definido em muitas operações pois evita o acesso à múltiplos registradores (poderia acabar causando *bugs* por conta dos *flags* de controle).

[03] - **A ULA gera um sinal que indicam quando ocorreu um *overflow* aritmético**. Todavia, devido ao intuito do projeto e do programa embarcado, esse sinal não altera em nada o funcionamento do projeto.

Instruções (binário e Assembly)

[04] - Os *bits don't care* recebem o valor 0 por padrão nas instruções em binário (evita conflitos nos *flags* de controle, acesso indevido a registradores, etc).

Apêndice B (resultado dos *testbenchs*)

Os *testbenchs* foram realizados no site `https://www.edaplayground.com/` (*Testbench*/*Design*: SystemVerilog/Verilog, Compilador/Simulador: Icarus Verilog 12.0)

Módulos individuais:

- **Controle:**

Op2	Op1	Op0	funct	reset	Sinais de controle										
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0
0	0	1	0	0	1	0	1	0	0	0	0	0	0	0	1
0	0	1	1	0	1	0	1	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	1	0	0	0	0	0	0	1	0
0	1	0	1	0	0	0	0	0	1	0	0	0	0	1	0
0	1	1	0	0	0	1	1	1	0	0	0	0	0	0	1
0	1	1	1	0	0	0	0	0	1	0	0	0	0	0	1
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1
1	0	0	1	0	0	0	0	0	0	0	1	0	0	0	1
1	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1
1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	1
1	1	0	0	0	1	0	0	0	0	1	0	0	0	0	0
1	1	0	1	0	1	0	0	0	0	1	0	0	0	0	0
1	1	1	0	0	1	1	0	0	0	0	0	1	0	0	1
1	1	1	1	0	1	0	0	0	0	0	0	0	1	0	1
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	1	0	0	0	0	0	0	0	1	0	1

- ***PC*:**

Time	Clock	Reset	End	endereco	enderecoSaida
0	0	1	0	5	0
5	1	1	0	5	0
10	0	0	0	5	0
15	1	0	0	5	5
20	0	0	0	10	5
25	1	0	0	10	10
30	0	0	0	20	10
35	1	0	0	20	20
40	0	0	1	30	20
45	1	0	1	30	20
50	0	0	1	40	20
55	1	0	1	40	20
60	0	1	1	40	0
65	1	1	1	40	0
70	0	0	0	40	0
75	1	0	0	40	40
80	0	0	0	40	40

- **Memória de dados:**

Endereço	Dado lido
0	01001000 (H)
1	01000101 (E)
2	01001100 (L)
3	01001100 (L)
4	01001111 (O)
5	00000000 ( )
0	01001000 (H)
1	01000101 (E)
2	01001100 (L)

```
     3    | 01001100 (L)
     4    | 01001111 (O)
     5    | 00000000 ( )
     6    | 01010111 (W)
     7    | 01001111 (O)
     8    | 01010010 (R)
     9    | 01001100 (L)
    10    | 01000100 (D)
     0    | 01010111 (W)
     1    | 01001111 (O)
     2    | 01010010 (R)
     3    | 01001100 (L)
     4    | 01000100 (D)
     5    | 00000000 ( )
     6    | 01010111 (W)
     7    | 01001111 (O)
     8    | 01010010 (R)
     9    | 01001100 (L)
    10    | 01000100 (D)
    20    | xxxxxxxx ( )
    20    | 01011010 (Z)
    20    | 01011010 (Z)
```

- **Memória de instruções:**

Endereço	OpCode	Reg1	Reg2	Funct	Imediato
0	000	10	10	1	101
4	001	00	01	1	011
10	011	01	10	0	100
19	110	11	00	1	001
54	001	00	01	1	011
77	111	00	00	0	000
81	100	00	00	0	000
83	111	00	00	1	001

- **Banco de registradores:**

Time	Clock	Reset	RegWrite	Reg1	Reg2	DadoW	Dado Reg1	Dado Reg2	ReIn	ReOut
0	0	1	0	00	00	00	00	00	00	fc
5	1	1	0	00	00	00	00	00	00	fc
10	0	0	1	00	00	aa	00	00	11	fc
15	1	0	1	00	00	aa	aa	aa	11	11
20	0	0	1	01	00	bb	00	aa	22	11
25	1	0	1	01	00	bb	bb	aa	22	22
30	0	0	1	10	00	cc	00	aa	33	22
35	1	0	1	10	00	cc	cc	aa	33	33
40	0	0	1	11	00	dd	00	aa	44	33
45	1	0	1	11	00	dd	dd	aa	44	44
50	0	0	0	00	01	dd	aa	bb	55	44
55	1	0	0	00	01	dd	aa	bb	55	55
60	0	0	0	10	11	dd	cc	dd	55	55
65	1	0	0	10	11	dd	cc	dd	55	55
70	0	0	0	00	00	ff	aa	aa	66	55
75	1	0	0	00	00	ff	aa	aa	66	66
80	0	0	0	00	00	ff	aa	aa	66	66
85	1	0	0	00	00	ff	aa	aa	66	66
90	0	0	1	00	00	ff	aa	aa	66	66
95	1	0	1	00	00	ff	ff	ff	66	66
100	0	0	1	00	00	ff	ff	ff	77	66
105	1	0	1	00	00	ff	ff	ff	77	77
110	0	0	1	00	00	ff	ff	ff	88	77
115	1	0	1	00	00	ff	ff	ff	88	88
120	0	1	1	00	00	ff	00	00	88	fc
125	1	1	1	00	00	ff	00	00	88	fc
130	0	0	1	00	00	ff	00	00	88	fc
135	1	0	1	00	00	ff	ff	ff	88	88
140	0	0	1	00	00	ff	ff	ff	88	88

- **ULA:**

A	B	S	overflow	reEntrada	reSaida	zero
10	20	30	0	aa	aa	0
255	1	0	1	55	55	0
30	15	15	1	bb	bb	0
50	25	75	0	ee	ee	0
10	5	50	0	ff	ff	0
128	2	0	1	66	66	0
40	40	0	1	11	01	0
40	41	0	0	22	00	0
10	20	0	0	33	01	0
30	20	0	1	44	00	0
25	204	0	1	cc	fc	1
30	25	0	0	dd	fc	0

Processador completo com programa embarcado:

- **Mensagem: HELLO - Cifra: 5**

Memoria de dados [0:6]:

Endereco	Valor	Decimal	Caractere
0	01001000	72	H
1	01000101	69	E
2	01001100	76	L
3	01001100	76	L
4	01001111	79	O
5	00000000	0	
6	xxxxxxxx	x	

PC	Instrução	Assembly	$c0	$c1	$c2	$c3	$re
0	00010101	sub $c2, $c2	0	0	0	0	-4
1	00000001	sub $c0, $c0	0	0	0	0	-4
2	00011111	sub $c3, $c3	0	0	0	0	-4
3	11100000	nop	0	0	0	0	-4
4	00100011	addi $c0, 3	0	0	0	0	-4
5	00100010	addi $c0, 2	3	0	0	0	-4
6	00100000	addi $c0, 0	5	0	0	0	-4
7	00100000	addi $c0, 0	5	0	0	0	-4
8	00100000	addi $c0, 0	5	0	0	0	-4
9	11100000	nop	5	0	0	0	-4
10	01101100	ld $c1, $c2	5	0	0	0	-4
11	00001000	add $c1, $c0	5	72	0	0	-4
12	10101000	beq $c1, $c0	5	77	0	0	-4
13	00000001	sub $c0, $c0	5	77	0	0	0
14	11100000	nop	0	77	0	0	0
15	00111011	addi $c3, 3	0	77	0	0	0
16	01011110	mult $c3, $c3	0	77	0	3	0
17	01011110	mult $c3, $c3	0	77	0	9	0
18	00111001	addi $c3, 1	0	77	0	81	0
19	11011001	result $c3, 1	0	77	0	82	0
20	00000001	sub $c0, $c0	0	77	0	82	-4
21	00011111	sub $c3, $c3	0	77	0	82	-4
22	11100000	nop	0	77	0	0	-4
23	00100011	addi $c0, 3	0	77	0	0	-4
24	00100011	addi $c0, 3	3	77	0	0	-4
25	00100011	addi $c0, 3	6	77	0	0	-4
26	00100001	addi $c0, 1	9	77	0	0	-4
27	00111011	addi $c3, 3	10	77	0	0	-4
28	00111011	addi $c3, 3	10	77	0	3	-4
29	00111011	addi $c3, 3	10	77	0	6	-4
30	01000110	mult $c0, $c3	10	77	0	9	-4
31	00100001	addi $c0, 1	90	77	0	9	-4
32	00011111	sub $c3, $c3	91	77	0	9	-4
33	10101001	slt $c1, $c0	91	77	0	0	-4
34	00000001	sub $c0, $c0	91	77	0	0	1
35	00011111	sub $c3, $c3	0	77	0	0	1
36	00111011	addi $c3, 3	0	77	0	0	1

37	00000110	add $c0, $c3	0	77	0	3	1
38	01011000	mult $c3, $c0	3	77	0	3	1
39	01011000	mult $c3, $c0	3	77	0	9	1
40	00011110	add $c3, $c3	3	77	0	27	1
41	00111101	addi $c3, 5	3	77	0	54	1
42	11011001	result $c3, 1	3	77	0	51	1
51	11100000	nop	3	77	0	51	-4
52	00000001	sub $c0, $c0	3	77	0	51	-4
53	00011111	sub $c3, $c3	0	77	0	51	-4
54	00100011	addi $c0, 3	0	77	0	0	-4
55	00100001	addi $c0, 1	3	77	0	0	-4
56	00011000	add $c3, $c0	4	77	0	0	-4
57	01000110	mult $c0, $c3	4	77	0	4	-4
58	01000110	mult $c0, $c3	16	77	0	4	-4
59	00100001	addi $c0, 1	64	77	0	4	-4
60	00011111	sub $c3, $c3	65	77	0	4	-4
61	10101001	slt $c1, $c0	65	77	0	0	-4
62	00000001	sub $c0, $c0	65	77	0	0	0
63	00111011	addi $c3, 3	0	77	0	0	0
64	01011110	mult $c3, $c3	0	77	0	3	0
65	01011110	mult $c3, $c3	0	77	0	9	0
66	00111101	addi $c3, 5	0	77	0	81	0
67	11011000	result $c3, 0	0	77	0	78	0
78	01101101	st $c1, $c2	0	77	0	78	-4
79	00110001	addi $c2, 1	0	77	0	78	-4
80	00100001	addi $c0, 1	0	77	1	78	-4
81	10000000	jr $c0	1	77	1	78	-4
1	00000001	sub $c0, $c0	1	77	1	78	-4
2	00011111	sub $c3, $c3	0	77	1	78	-4
3	11100000	nop	0	77	1	0	-4
4	00100011	addi $c0, 3	0	77	1	0	-4
5	00100010	addi $c0, 2	3	77	1	0	-4
6	00100000	addi $c0, 0	5	77	1	0	-4
7	00100000	addi $c0, 0	5	77	1	0	-4
8	00100000	addi $c0, 0	5	77	1	0	-4
9	11100000	nop	5	77	1	0	-4
10	01101100	ld $c1, $c2	5	77	1	0	-4
11	00001000	add $c1, $c0	5	69	1	0	-4
12	10101000	beq $c1, $c0	5	74	1	0	-4
13	00000001	sub $c0, $c0	5	74	1	0	0
14	11100000	nop	0	74	1	0	0
15	00111011	addi $c3, 3	0	74	1	0	0
16	01011110	mult $c3, $c3	0	74	1	3	0
17	01011110	mult $c3, $c3	0	74	1	9	0
18	00111001	addi $c3, 1	0	74	1	81	0
19	11011001	result $c3, 1	0	74	1	82	0
20	00000001	sub $c0, $c0	0	74	1	82	-4
21	00011111	sub $c3, $c3	0	74	1	82	-4
22	11100000	nop	0	74	1	0	-4
23	00100011	addi $c0, 3	0	74	1	0	-4
24	00100011	addi $c0, 3	3	74	1	0	-4
25	00100011	addi $c0, 3	6	74	1	0	-4
26	00100001	addi $c0, 1	9	74	1	0	-4
27	00111011	addi $c3, 3	10	74	1	0	-4
28	00111011	addi $c3, 3	10	74	1	3	-4
29	00111011	addi $c3, 3	10	74	1	6	-4
30	01000110	mult $c0, $c3	10	74	1	9	-4
31	00100001	addi $c0, 1	90	74	1	9	-4
32	00011111	sub $c3, $c3	91	74	1	9	-4
33	10101001	slt $c1, $c0	91	74	1	0	-4
34	00000001	sub $c0, $c0	91	74	1	0	1
35	00011111	sub $c3, $c3	0	74	1	0	1
36	00111011	addi $c3, 3	0	74	1	0	1
37	00000110	add $c0, $c3	0	74	1	3	1
38	01011000	mult $c3, $c0	3	74	1	3	1
39	01011000	mult $c3, $c0	3	74	1	9	1
40	00011110	add $c3, $c3	3	74	1	27	1
41	00111101	addi $c3, 5	3	74	1	54	1
42	11011001	result $c3, 1	3	74	1	51	1
51	11100000	nop	3	74	1	51	-4

52	00000001	sub $c0, $c0	3	74	1	51	-4
53	00011111	sub $c3, $c3	0	74	1	51	-4
54	00100011	addi $c0, 3	0	74	1	0	-4
55	00100001	addi $c0, 1	3	74	1	0	-4
56	00011000	add $c3, $c0	4	74	1	0	-4
57	01000110	mult $c0, $c3	4	74	1	4	-4
58	01000110	mult $c0, $c3	16	74	1	4	-4
59	00100001	addi $c0, 1	64	74	1	4	-4
60	00011111	sub $c3, $c3	65	74	1	4	-4
61	10101001	slt $c1, $c0	65	74	1	0	-4
62	00000001	sub $c0, $c0	65	74	1	0	0
63	00111011	addi $c3, 3	0	74	1	0	0
64	01011110	mult $c3, $c3	0	74	1	3	0
65	01011110	mult $c3, $c3	0	74	1	9	0
66	00111101	addi $c3, 5	0	74	1	81	0
67	11011000	result $c3, 0	0	74	1	78	0
78	01101101	st $c1, $c2	0	74	1	78	-4
79	00110001	addi $c2, 1	0	74	1	78	-4
80	00100001	addi $c0, 1	0	74	2	78	-4
81	10000000	jr $c0	1	74	2	78	-4
1	00000001	sub $c0, $c0	1	74	2	78	-4
2	00011111	sub $c3, $c3	0	74	2	78	-4
3	11100000	nop	0	74	2	0	-4
4	00100011	addi $c0, 3	0	74	2	0	-4
5	00100010	addi $c0, 2	3	74	2	0	-4
6	00100000	addi $c0, 0	5	74	2	0	-4
7	00100000	addi $c0, 0	5	74	2	0	-4
8	00100000	addi $c0, 0	5	74	2	0	-4
9	11100000	nop	5	74	2	0	-4
10	01101100	ld $c1, $c2	5	74	2	0	-4
11	00001000	add $c1, $c0	5	76	2	0	-4
12	10101000	beq $c1, $c0	5	81	2	0	-4
13	00000001	sub $c0, $c0	5	81	2	0	0
14	11100000	nop	0	81	2	0	0
15	00111011	addi $c3, 3	0	81	2	0	0
16	01011110	mult $c3, $c3	0	81	2	3	0
17	01011110	mult $c3, $c3	0	81	2	9	0
18	00111001	addi $c3, 1	0	81	2	81	0
19	11011001	result $c3, 1	0	81	2	82	0
20	00000001	sub $c0, $c0	0	81	2	82	-4
21	00011111	sub $c3, $c3	0	81	2	82	-4
22	11100000	nop	0	81	2	0	-4
23	00100011	addi $c0, 3	0	81	2	0	-4
24	00100011	addi $c0, 3	3	81	2	0	-4
25	00100011	addi $c0, 3	6	81	2	0	-4
26	00100001	addi $c0, 1	9	81	2	0	-4
27	00111011	addi $c3, 3	10	81	2	0	-4
28	00111011	addi $c3, 3	10	81	2	3	-4
29	00111011	addi $c3, 3	10	81	2	6	-4
30	01000110	mult $c0, $c3	10	81	2	9	-4
31	00100001	addi $c0, 1	90	81	2	9	-4
32	00011111	sub $c3, $c3	91	81	2	9	-4
33	10101001	slt $c1, $c0	91	81	2	0	-4
34	00000001	sub $c0, $c0	91	81	2	0	1
35	00011111	sub $c3, $c3	0	81	2	0	1
36	00111011	addi $c3, 3	0	81	2	0	1
37	00000110	add $c0, $c3	0	81	2	3	1
38	01011000	mult $c3, $c0	3	81	2	3	1
39	01011000	mult $c3, $c0	3	81	2	9	1
40	00011110	add $c3, $c3	3	81	2	27	1
41	00111101	addi $c3, 5	3	81	2	54	1
42	11011001	result $c3, 1	3	81	2	51	1
51	11100000	nop	3	81	2	51	-4
52	00000001	sub $c0, $c0	3	81	2	51	-4
53	00011111	sub $c3, $c3	0	81	2	51	-4
54	00100011	addi $c0, 3	0	81	2	0	-4
55	00100001	addi $c0, 1	3	81	2	0	-4
56	00011000	add $c3, $c0	4	81	2	0	-4
57	01000110	mult $c0, $c3	4	81	2	4	-4
58	01000110	mult $c0, $c3	16	81	2	4	-4

59	00100001	addi $c0, 1	64	81	2	4	-4
60	00011111	sub $c3, $c3	65	81	2	4	-4
61	10101001	slt $c1, $c0	65	81	2	0	-4
62	00000001	sub $c0, $c0	65	81	2	0	0
63	00111011	addi $c3, 3	0	81	2	0	0
64	01011110	mult $c3, $c3	0	81	2	3	0
65	01011110	mult $c3, $c3	0	81	2	9	0
66	00111101	addi $c3, 5	0	81	2	81	0
67	11011000	result $c3, 0	0	81	2	78	0
78	01101101	st $c1, $c2	0	81	2	78	-4
79	00110001	addi $c2, 1	0	81	2	78	-4
80	00100001	addi $c0, 1	0	81	3	78	-4
81	10000000	jr $c0	1	81	3	78	-4
1	00000001	sub $c0, $c0	1	81	3	78	-4
2	00011111	sub $c3, $c3	0	81	3	78	-4
3	11100000	nop	0	81	3	0	-4
4	00100011	addi $c0, 3	0	81	3	0	-4
5	00100010	addi $c0, 2	3	81	3	0	-4
6	00100000	addi $c0, 0	5	81	3	0	-4
7	00100000	addi $c0, 0	5	81	3	0	-4
8	00100000	addi $c0, 0	5	81	3	0	-4
9	11100000	nop	5	81	3	0	-4
10	01101100	ld $c1, $c2	5	81	3	0	-4
11	00001000	add $c1, $c0	5	76	3	0	-4
12	10101000	beq $c1, $c0	5	81	3	0	-4
13	00000001	sub $c0, $c0	5	81	3	0	0
14	11100000	nop	0	81	3	0	0
15	00111011	addi $c3, 3	0	81	3	0	0
16	01011110	mult $c3, $c3	0	81	3	3	0
17	01011110	mult $c3, $c3	0	81	3	9	0
18	00111001	addi $c3, 1	0	81	3	81	0
19	11011001	result $c3, 1	0	81	3	82	0
20	00000001	sub $c0, $c0	0	81	3	82	-4
21	00011111	sub $c3, $c3	0	81	3	82	-4
22	11100000	nop	0	81	3	0	-4
23	00100011	addi $c0, 3	0	81	3	0	-4
24	00100011	addi $c0, 3	3	81	3	0	-4
25	00100011	addi $c0, 3	6	81	3	0	-4
26	00100001	addi $c0, 1	9	81	3	0	-4
27	00111011	addi $c3, 3	10	81	3	0	-4
28	00111011	addi $c3, 3	10	81	3	3	-4
29	00111011	addi $c3, 3	10	81	3	6	-4
30	01000110	mult $c0, $c3	10	81	3	9	-4
31	00100001	addi $c0, 1	90	81	3	9	-4
32	00011111	sub $c3, $c3	91	81	3	9	-4
33	10101001	slt $c1, $c0	91	81	3	0	-4
34	00000001	sub $c0, $c0	91	81	3	0	1
35	00011111	sub $c3, $c3	0	81	3	0	1
36	00111011	addi $c3, 3	0	81	3	0	1
37	00000110	add $c0, $c3	0	81	3	3	1
38	01011000	mult $c3, $c0	3	81	3	3	1
39	01011000	mult $c3, $c0	3	81	3	9	1
40	00011110	add $c3, $c3	3	81	3	27	1
41	00111101	addi $c3, 5	3	81	3	54	1
42	11011001	result $c3, 1	3	81	3	51	1
51	11100000	nop	3	81	3	51	-4
52	00000001	sub $c0, $c0	3	81	3	51	-4
53	00011111	sub $c3, $c3	0	81	3	51	-4
54	00100011	addi $c0, 3	0	81	3	0	-4
55	00100001	addi $c0, 1	3	81	3	0	-4
56	00011000	add $c3, $c0	4	81	3	0	-4
57	01000110	mult $c0, $c3	4	81	3	4	-4
58	01000110	mult $c0, $c3	16	81	3	4	-4
59	00100001	addi $c0, 1	64	81	3	4	-4
60	00011111	sub $c3, $c3	65	81	3	4	-4
61	10101001	slt $c1, $c0	65	81	3	0	-4
62	00000001	sub $c0, $c0	65	81	3	0	0
63	00111011	addi $c3, 3	0	81	3	0	0
64	01011110	mult $c3, $c3	0	81	3	3	0
65	01011110	mult $c3, $c3	0	81	3	9	0

66	00111101	addi $c3, 5	0	81	3	81	0
67	11011000	result $c3, 0	0	81	3	78	0
78	01101101	st $c1, $c2	0	81	3	78	-4
79	00110001	addi $c2, 1	0	81	3	78	-4
80	00100001	addi $c0, 1	0	81	4	78	-4
81	10000000	jr $c0	1	81	4	78	-4
1	00000001	sub $c0, $c0	1	81	4	78	-4
2	00011111	sub $c3, $c3	0	81	4	78	-4
3	11100000	nop	0	81	4	0	-4
4	00100011	addi $c0, 3	0	81	4	0	-4
5	00100010	addi $c0, 2	3	81	4	0	-4
6	00100000	addi $c0, 0	5	81	4	0	-4
7	00100000	addi $c0, 0	5	81	4	0	-4
8	00100000	addi $c0, 0	5	81	4	0	-4
9	11100000	nop	5	81	4	0	-4
10	01101100	ld $c1, $c2	5	81	4	0	-4
11	00001000	add $c1, $c0	5	79	4	0	-4
12	10101000	beq $c1, $c0	5	84	4	0	-4
13	00000001	sub $c0, $c0	5	84	4	0	0
14	11100000	nop	0	84	4	0	0
15	00111011	addi $c3, 3	0	84	4	0	0
16	01011110	mult $c3, $c3	0	84	4	3	0
17	01011110	mult $c3, $c3	0	84	4	9	0
18	00111001	addi $c3, 1	0	84	4	81	0
19	11011001	result $c3, 1	0	84	4	82	0
20	00000001	sub $c0, $c0	0	84	4	82	-4
21	00011111	sub $c3, $c3	0	84	4	82	-4
22	11100000	nop	0	84	4	0	-4
23	00100011	addi $c0, 3	0	84	4	0	-4
24	00100011	addi $c0, 3	3	84	4	0	-4
25	00100011	addi $c0, 3	6	84	4	0	-4
26	00100001	addi $c0, 1	9	84	4	0	-4
27	00111011	addi $c3, 3	10	84	4	0	-4
28	00111011	addi $c3, 3	10	84	4	3	-4
29	00111011	addi $c3, 3	10	84	4	6	-4
30	01000110	mult $c0, $c3	10	84	4	9	-4
31	00100001	addi $c0, 1	90	84	4	9	-4
32	00011111	sub $c3, $c3	91	84	4	9	-4
33	10101001	slt $c1, $c0	91	84	4	0	-4
34	00000001	sub $c0, $c0	91	84	4	0	1
35	00011111	sub $c3, $c3	0	84	4	0	1
36	00111011	addi $c3, 3	0	84	4	0	1
37	00000110	add $c0, $c3	0	84	4	3	1
38	01011000	mult $c3, $c0	3	84	4	3	1
39	01011000	mult $c3, $c0	3	84	4	9	1
40	00011110	add $c3, $c3	3	84	4	27	1
41	00111101	addi $c3, 5	3	84	4	54	1
42	11011001	result $c3, 1	3	84	4	51	1
51	11100000	nop	3	84	4	51	-4
52	00000001	sub $c0, $c0	3	84	4	51	-4
53	00011111	sub $c3, $c3	0	84	4	51	-4
54	00100011	addi $c0, 3	0	84	4	0	-4
55	00100001	addi $c0, 1	3	84	4	0	-4
56	00011000	add $c3, $c0	4	84	4	0	-4
57	01000110	mult $c0, $c3	4	84	4	4	-4
58	01000110	mult $c0, $c3	16	84	4	4	-4
59	00100001	addi $c0, 1	64	84	4	4	-4
60	00011111	sub $c3, $c3	65	84	4	4	-4
61	10101001	slt $c1, $c0	65	84	4	0	-4
62	00000001	sub $c0, $c0	65	84	4	0	0
63	00111011	addi $c3, 3	0	84	4	0	0
64	01011110	mult $c3, $c3	0	84	4	3	0
65	01011110	mult $c3, $c3	0	84	4	9	0
66	00111101	addi $c3, 5	0	84	4	81	0
67	11011000	result $c3, 0	0	84	4	78	0
78	01101101	st $c1, $c2	0	84	4	78	-4
79	00110001	addi $c2, 1	0	84	4	78	-4
80	00100001	addi $c0, 1	0	84	5	78	-4
81	10000000	jr $c0	1	84	5	78	-4
1	00000001	sub $c0, $c0	1	84	5	78	-4

```
 2 |  00011111 | sub $c3, $c3    |   0   |  84   |   5   |  78   |  -4
 3 |  11100000 | nop             |   0   |  84   |   5   |   0   |  -4
 4 |  00100011 | addi $c0, 3     |   0   |  84   |   5   |   0   |  -4
 5 |  00100010 | addi $c0, 2     |   3   |  84   |   5   |   0   |  -4
 6 |  00100000 | addi $c0, 0     |   5   |  84   |   5   |   0   |  -4
 7 |  00100000 | addi $c0, 0     |   5   |  84   |   5   |   0   |  -4
 8 |  00100000 | addi $c0, 0     |   5   |  84   |   5   |   0   |  -4
 9 |  11100000 | nop             |   5   |  84   |   5   |   0   |  -4
10 |  01101100 | ld $c1, $c2     |   5   |  84   |   5   |   0   |  -4
11 |  00001000 | add $c1, $c0    |   5   |   0   |   5   |   0   |  -4
12 |  10101000 | beq $c1, $c0    |   5   |   5   |   5   |   0   |  -4
13 |  00000001 | sub $c0, $c0    |   5   |   5   |   5   |   0   |   1
14 |  11100000 | nop             |   0   |   5   |   5   |   0   |   1
15 |  00111011 | addi $c3, 3     |   0   |   5   |   5   |   0   |   1
16 |  01011110 | mult $c3, $c3   |   0   |   5   |   5   |   3   |   1
17 |  01011110 | mult $c3, $c3   |   0   |   5   |   5   |   9   |   1
18 |  00111001 | addi $c3, 1     |   0   |   5   |   5   |  81   |   1
19 |  11011001 | result $c3, 1   |   0   |   5   |   5   |  82   |   1
82 |  11100000 | nop             |   0   |   5   |   5   |  82   |  -4
83 |  11100001 | hlt             |   0   |   5   |   5   |  82   |  -4
Memoria de dados [0:6]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01001101  |    77   |     M
    1    |  01001010  |    74   |     J
    2    |  01010001  |    81   |     Q
    3    |  01010001  |    81   |     Q
    4    |  01010100  |    84   |     T
    5    |  00000000  |     0   |
    6    |  xxxxxxxx  |     x   |
```

- ***testbench*** **resumido (Mensagem: BATATA - Cifra: 15 -** `show_data_memory(0, 7);`**)**

```
Memoria de dados [0:7]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01000010  |    66   |     B
    1    |  01000001  |    65   |     A
    2    |  01010100  |    84   |     T
    3    |  01000001  |    65   |     A
    4    |  01010100  |    84   |     T
    5    |  01000001  |    65   |     A
    6    |  00000000  |     0   |
    7    |  xxxxxxxx  |     x   |
Memoria de dados [0:7]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01010001  |    81   |     Q
    1    |  01010000  |    80   |     P
    2    |  01001001  |    73   |     I
    3    |  01010000  |    80   |     P
    4    |  01001001  |    73   |     I
    5    |  01010000  |    80   |     P
    6    |  00000000  |     0   |
    7    |  xxxxxxxx  |     x   |
```

- ***testbench*** **resumido (Mensagem: GALOFRITO - Cifra: -15 -** `show_data_memory(0, 10);`**)**

```
Memoria de dados [0:10]:
Endereco |    Valor   | Decimal | Caractere
    0    |  01000111  |    71   |     G
    1    |  01000001  |    65   |     A
    2    |  01001100  |    76   |     L
    3    |  01001111  |    79   |     O
    4    |  01000110  |    70   |     F
    5    |  01010010  |    82   |     R
    6    |  01001001  |    73   |     I
    7    |  01010100  |    84   |     T
    8    |  01001111  |    79   |     O
```

```
     9    |  00000000  |     0   |
    10    |  xxxxxxxx  |     x   |
Memoria de dados [0:10]:
Endereco |    Valor   | Decimal | Caractere
     0    |  01010010  |    82   |     R
     1    |  01001100  |    76   |     L
     2    |  01010111  |    87   |     W
     3    |  01011010  |    90   |     Z
     4    |  01010001  |    81   |     Q
     5    |  01000011  |    67   |     C
     6    |  01010100  |    84   |     T
     7    |  01000101  |    69   |     E
     8    |  01011010  |    90   |     Z
     9    |  00000000  |     0   |
    10    |  xxxxxxxx  |     x   |
```

Cifra com elementos após o *EOF*:

- ***testbench*** **resumido (Mensagem: GALO FRITO - Cifra: -15 -** `show_data_memory(0, 11);`**)**

```
Memoria de dados [0:11]:
Endereco |    Valor   | Decimal | Caractere
     0    |  01000111  |    71   |     G
     1    |  01000001  |    65   |     A
     2    |  01001100  |    76   |     L
     3    |  01001111  |    79   |     O
     4    |  00000000  |     0   |
     5    |  01000110  |    70   |     F
     6    |  01010010  |    82   |     R
     7    |  01001001  |    73   |     I
     8    |  01010100  |    84   |     T
     9    |  01001111  |    79   |     O
    10    |  00000000  |     0   |
    11    |  xxxxxxxx  |     x   |
Memoria de dados [0:11]:
Endereco |    Valor   | Decimal | Caractere
     0    |  01010010  |    82   |     R
     1    |  01001100  |    76   |     L
     2    |  01010111  |    87   |     W
     3    |  01011010  |    90   |     Z
     4    |  00000000  |     0   |
     5    |  01000110  |    70   |     F
     6    |  01010010  |    82   |     R
     7    |  01001001  |    73   |     I
     8    |  01010100  |    84   |     T
     9    |  01001111  |    79   |     O
    10    |  00000000  |     0   |
    11    |  xxxxxxxx  |     x   |
```

Apêndice C (programa embarcado binário e Assembly)

```
// Para tornar o código programável, sem que a mudança de cifra alterasse a quantidade de linhas (já que as
↪  somas com imediato só podem ser feitas de 3 em 3 e os jumps são pra uma linha específica), faz-se uma soma
↪  de 5 linhas sempre (que vai de -15 até 15, que cobre todo o range)
 0- 00010101  // sub $c2, $c2 (zera os registradores)
 1- 00000001  // sub $c0, $c0
 2- 00011111  // sub $c3, $c3
 3- 11100000  // nop
 4- 00100011  // addi $c0, 3 (incrementa o valor da cifra)
 5- 00100010  // addi $c0, 2 (como o resto é 0, a cifra é 5)
 6- 00100000  // addi $c0, 0
 7- 00100000  // addi $c0, 0
 8- 00100000  // addi $c0, 0
 9- 11100000  // nop
10- 01101100  // ld $c1, $c2 (recebe o valor armazenado na posição indicada por $c2 e o copia em $c1)
11- 00001000  // add $c1, $c0 (adiciona a cifra ao valor)
12- 10101000  // beq $c1, $c0 (testa se chegou ao EOF (0))
13- 00000001  // sub $c0, $c0
14- 11100000  // nop
15- 00111011  // addi $c3, 3
16- 01011110  // mult $c3, $c3
17- 01011110  // mult $c3, $c3
18- 00111001  // addi $c3, 1 // ($c3 == 82)
19- 11011001  // result $c3, 1 (se chegou ao EOF, vai para o fim do código)
20- 00000001  // sub $c0, $c0
21- 00011111  // sub $c3, $c3
22- 11100000  // nop
23- 00100011  // addi $c0, 3
24- 00100011  // addi $c0, 3
25- 00100011  // addi $c0, 3
26- 00100001  // addi $c0, 1
27- 00111011  // addi $c3, 3
28- 00111011  // addi $c3, 3
29- 00111011  // addi $c3, 3
30- 01000110  // mult $c0, $c3
31- 00100001  // addi $c0, 1 ($c0 == 91)
32- 00011111  // sub $c3, $c3
33- 10101001  // slt $c1, $c0 (Verifica se $c1 < 91 (91 = 'Z') ? $re = 1 : $re = 0)
34- 00000001  // sub $c0, $c0
35- 00011111  // sub $c3, $c3
36- 00111011  // addi $c3, 3
37- 00000110  // add $c0, $c3
38- 01011000  // mult $c3, $c0
39- 01011000  // mult $c3, $c0
40- 00011110  // add $c3, $c3
41- 00111101  // $addi $c3, -3 ($c3 == 51)
42- 11011001  // result $c3, 1  (se a letra está no range, pula a operação de correção)
43- 00011111  // sub $c3, $c3
44- 00000001  // sub $c0, $c0
45- 00100011  // addi $c0, 3
46- 00111011  // addi $c3, 3
47- 01000110  // mult $c0, $c3
48- 01000110  // mult $c0, $c3
49- 00100111  // addi $c0, -1 ($c0 == 26)
50- 00001001  // sub $c1, $c0 (o valor é decrementado para caber no range)
51- 11100000  // nop
52- 00000001  // sub $c0, $c0
53- 00011111  // sub $c3, $c3
54- 00100011  // addi $c0, 3
55- 00100001  // addi $c0, 1
56- 00011000  // add $c3, $c0
57- 01000110  // mult $c0, $c3
58- 01000110  // mult $c0, $c3
59- 00100001  // addi $c0, 1 ($c0 == 65)
60- 00011111  // sub $c3, $c3
61- 10101001  // slt $c1, $c0 (Verifica se $c1 < 65 (65 = 'A') ? $re = 1 : $re = 0)
62- 00000001  // sub $c0, $c0
63- 00111011  // addi $c3, 3
64- 01011110  // mult $c3, $c3
```

```
65- 01011110  // mult $c3, $c3
66- 00111101  // addi $c3, -3 (c3 == 78)
67- 11011000  // result $c3, 0 (se a letra está no range, pula a operação de correção)
68- 00011111  // sub $c3, $c3
69- 00000001  // sub $c0, $c0
70- 00100011  // addi $c0, 3
71- 00111011  // addi $c3, 3
72- 01000110  // mult $c0, $c3
73- 01000110  // mult $c0, $c3
74- 00100111  // addi $c0, -1 ($c0 == 26)
75- 00001000  // add $c1, $c0 (o valor é incrementado para caber no range)
76- 00000001  // sub $c0, $c0
77- 11100000  // nop
78- 01101101  // st $c1, $c2 (armazena o valor de $c1 no endereço indicado por $c2)
79- 00110001  // addi $c2, 1 (passa para o próximo caracter)
80- 00100001  // addi $c0, 1
81- 10000000  // jr $c0 (volta para o início do código)
82- 11100000  // nop
83- 11100001  // hlt (encerra o programa)
```